Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

ОТЧЁТ
по лабораторной работе №7
по дисциплине
«Информационные технологии и веб-программирование»
Тема: Реализация реального времени с WebSocket

Выполнила: студентка группы 320604
Хаджинова К.А.

Проверил: Хаджинова Н.В.

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 3
3 ХОД РАБОТЫ 3
3.1 Схема работы 3
3.2 Выбор клиента и структура проекта 4
3.3 События 5
3.4 Дополнительные функции 5
3.5 Проверка 6
3.5.1 Проверка в двух окнах браузера 7
3.6 Проверка вводимых значений и ограничения 13
3.7 Ответы на контрольные вопросы 13
ЗАКЛЮЧЕНИЕ 15
ПРИЛОЖЕНИЕ А Листинг server.js 16
ПРИЛОЖЕНИЕ Б Листинг app.js 17
ПРИЛОЖЕНИЕ В Листинг socket.js 20
ПРИЛОЖЕНИЕ Г Листинг validate.js 26
ПРИЛОЖЕНИЕ Д Листинг history.js 27
ПРИЛОЖЕНИЕ Е Листинг rate-limit.js 28
ПРИЛОЖЕНИЕ Ж Листинг client.js 29
ПРИЛОЖЕНИЕ И Листинг index.html 36

1 ЦЕЛЬ РАБОТЫ

Освоить технологию WebSocket для двусторонней связи в реальном времени и библиотеку Socket.IO. Работа выполнена для проекта DSLKit, веб-редактора диаграмм как кода: сервер даёт каждой диаграмме свою комнату, в которой пользователи обмениваются сообщениями, получают уведомления о входе и выходе и видят, кто сейчас открыл диаграмму и кто её редактирует.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Ограничения HTTP. В модели запрос-ответ сервер не может сам отправить данные клиенту. Чтобы узнать о новом сообщении, клиент вынужден опрашивать сервер, что создаёт лишнюю нагрузку и задержку.

WebSocket. Соединение начинается обычным HTTP-запросом с заголовком `Upgrade: websocket`. Сервер отвечает кодом 101, после чего то же TCP-соединение остаётся открытым, и обе стороны отправляют сообщения в любой момент без новых заголовков.

Socket.IO. Библиотека поверх WebSocket добавляет именованные события с данными, подтверждения доставки (acknowledgement), комнаты, автоматическое переподключение и запасной вариант с long polling, если WebSocket недоступен.

Рассылка. Различаются три способа отправки: `socket.emit` только этому клиенту, `socket.broadcast.emit` всем, кроме него, `io.emit` всем подключённым. Метод `to(room)` ограничивает получателей комнатой.

3 ХОД РАБОТЫ

3.1 Схема работы

Взаимодействие клиента и сервера описано последовательностью шагов.

1. Клиент открывает страницу, вводит имя и вызывает `io({ auth: { username } })`. Сервер проверяет имя в промежуточном обработчике `io.use` и либо отклоняет подключение, либо принимает его.
2. После соединения сервер отправляет клиенту `welcome` и сообщает остальным `user:online`.
3. Клиент отправляет `room:join` с названием комнаты диаграммы. Сервер добавляет сокет в комнату, оповещает остальных событием `room:joined`, рассылает `presence:update` и подтверждает вход историей последних сообщений.

4. Клиент отправляет `chat:message`, сервер сохраняет сообщение в историю и рассылает его всем в комнате. Событие `typing` уходит остальным участникам, `editing` обновляет список.
5. При закрытии вкладки сервер получает `disconnect`, убирает пользователя из комнат и рассылает `room:left` и `user:offline`.

Последовательность обмена показана на рисунке 1: вверху три участника, ниже по стрелкам идут события в порядке выполнения, слева направо отправляет клиент A, справа налево приходит ответ сервера.

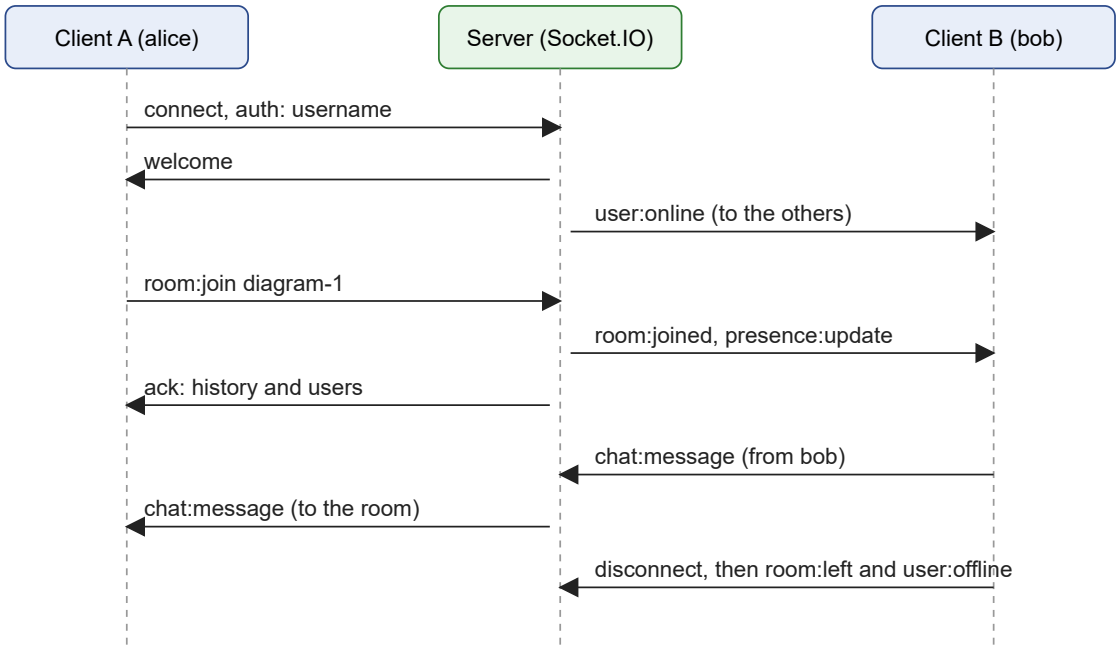


Рисунок 1 – Схема обмена событиями при подключении, входе в комнату, сообщении и отключении

3.2 Выбор клиента и структура проекта

Для клиента выбраны чистые HTML и JavaScript. Страница с чатом небольшая, сборщик ей не нужен. Express раздаёт её из папки `public` с того же адреса, что и сервер, поэтому заголовки CORS не нужны и сервер их не отправляет, а для проверки достаточно открыть один адрес в двух окнах. Протокол событий не зависит от клиента, и React-компонент для основного проекта сможет использовать его без изменений. Структура проекта показана в таблице 1.

Таблица 1 – Структура проекта

Файл	**Назначение**
`server.js`	Читает `.env`, запускает сервер на порту из `PORT` или 3000

Файл	Назначение
src/app.js	Функция createServer: Express, статические файлы и Socket.IO
src/socket.js	Обработчики всех событий
src/validate.js	Проверка имени, названия комнаты и текста сообщения
src/history.js	История сообщений: в памяти или в MongoDB (заменяемое хранилище)
src/token.js	Подписанный токен входа (HMAC-SHA256, срок действия)
src/rate-limit.js	Ограничение частоты сообщений
public/	Клиент: страница, скрипт и стили
tests/	Тесты на socket.io-client

3.3 События

События и их получатели перечислены в таблице 2.

Таблица 2 – События протокола

Событие	Направление	Получатели
welcome	сервер → клиент	только подключившийся
user:online	сервер → клиент	все, кроме подключившегося
user:offline	сервер → клиент	все остальные
room:join	клиент → сервер	ответ через acknowledgement
room:joined	сервер → клиент	остальные в комнате
room:leave	клиент → сервер	ответ через acknowledgement
room:left	сервер → клиент	все в комнате
presence:update	сервер → клиент	все в комнате, со списком людей и editing
chat:message	в обе стороны	все в комнате, включая автора
private:message	в обе стороны	только адресат, выбранный по socket.id
typing	в обе стороны	остальные в комнате, сообщение не хранится
editing	клиент → сервер	обновляет список через presence:update

3.4 Дополнительные функции

Выполнены дополнительные функции из условия и несколько своих.

Комнаты. Название комнаты соответствует диаграмме, например diagram-1. Сообщения, уведомления и список людей не выходят за пределы комнаты. Название проверяется регулярным выражением, чтобы клиент не создал комнату со служебным именем.

Кто редактирует диаграмму. Сервер хранит для каждой комнаты список пользователей с флагом `editing`. Любое изменение состава или флага рассылается событием `presence:update`, а клиент показывает пометку «editing» рядом с именем.

История. Последние 50 сообщений комнаты приходят новому участнику в ответе на `room:join`. Обработчики используют у хранилища только методы `add` и `list` и ждут их через `await`, поэтому хранилище подменяется. По умолчанию история лежит в памяти. Если задана переменная `MONGODB_URI`, `server.js` подключает драйвер `mongodb`, а `createMongoHistory` пишет сообщения в коллекцию `messages` и читает последние 50 по комнате, так что история переживает перезапуск. Если база недоступна, сервер отвечает ошибкой подтверждения и не падает. Тесты используют небольшую подставную коллекцию, настоящая база в тестах и при подготовке отчёта не запускалась.

Вход по токену. Маршрут `POST /login` с телом `{ "username": "..." }` возвращает подписанный токен (HMAC-SHA256, срок действия один час). Сокет подключается с `auth: { token }`, имя берётся из токена. Секрет читается из переменной `TOKEN_SECRET`, для локального запуска и тестов есть значение по умолчанию. При `REQUIRE_TOKEN=1` сокеты без токена отклоняются. Подделанные, чужие и просроченные токены отклоняются.

Личные сообщения. Событие `private:message` принимает `socket.id` получателя, доставляет текст только ему и возвращает отправителю подтверждение с копией сообщения.

Уведомления о наборе текста. Событие `typing` отправляется остальным участникам комнаты и не сохраняется. Клиент отправляет `true` при вводе и `false` через секунду после последней клавиши.

Подтверждения и ограничения. Все события с ответом возвращают `{ ok, error }`. Длина текста, частота сообщений и размер пакета заданы в файле `limits.json` (подробнее в разделе 3.6). Клиент показывает текст через `textContent`, поэтому сообщение не может внедрить разметку.

3.5 Проверка

Работа проверена автоматическими тестами: 102 теста запускают настоящий сервер на свободном порту и подключают несколько клиентов `socket.io-client` по WebSocket. Результаты по сценариям сведены в таблице 3. Ручная проверка выполняется в двух окнах браузера по шагам из файла README.

Таблица 3 – Сценарии автоматической проверки

Сценарий	**Ожидаемый результат**
Подключение с корректным именем	приходит `welcome` со списком подключённых
Пустое, неверное или занятое имя	подключение отклонено с понятным сообщением
Подключение и отключение второго клиента	первый получает `user:online` и `user:offline`
Вход в комнату	история, список людей, `room:joined` у остальных
Сообщение в комнате	получают все в комнате, в другой комнате ничего не приходит
Сообщение вне комнаты, пустое, длинное, частое	отказ с текстом ошибки в acknowledgement
Четвёртое сообщение подряд при лимите три	отказ `Too many messages, slow down`
Новый участник после четырёх сообщений	получает только последние три (лимит истории в тесте)
Выход и отключение	`room:left` и обновлённый список людей
Набор текста	событие получают другие, отправитель не получает
Флаг редактирования	список показывает автора с `editing`
Личное сообщение	получает только адресат, неизвестный адресат даёт отказ
Раздача клиента и `/health`	страница и ответ `ok` отдаются

3.5.1 Проверка в двух окнах браузера

Ручная проверка выполнена в нескольких окнах браузера, каждое окно это отдельный клиент (Алиса, Боб, Кэрол). Снимки сделаны скриптом `shots/shots.mjs` в настоящем браузере Chromium, который подключается к запущенному серверу. Порядок действий: подключение двух клиентов, вход в комнату, обмен сообщениями, проверка изоляции комнат, признак редактирования, ошибка проверки, выход и отключение. Результаты показаны на рисунках 2–11.

На рисунке 2 показано: окно Алисы: приветствие и уведомления о подключении Боба.

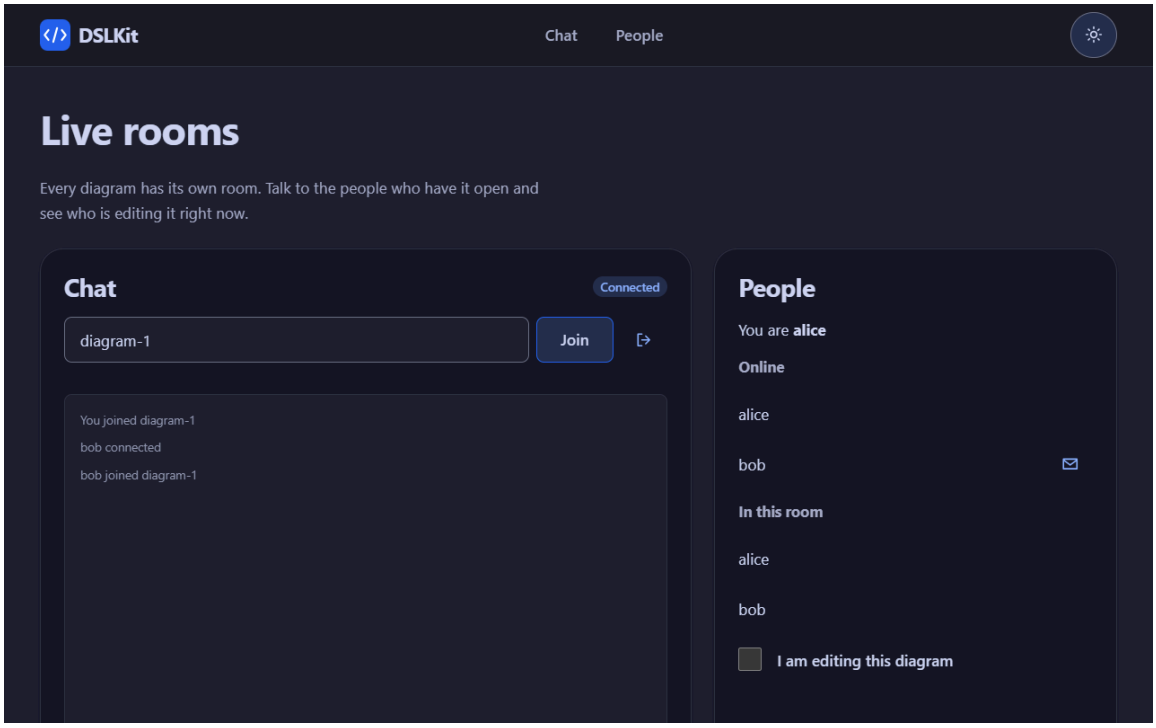


Рисунок 2 – Окно Алисы: приветствие и уведомления о подключении Боба

На рисунке 3 показано: окно Боба после входа в комнату диаграммы.

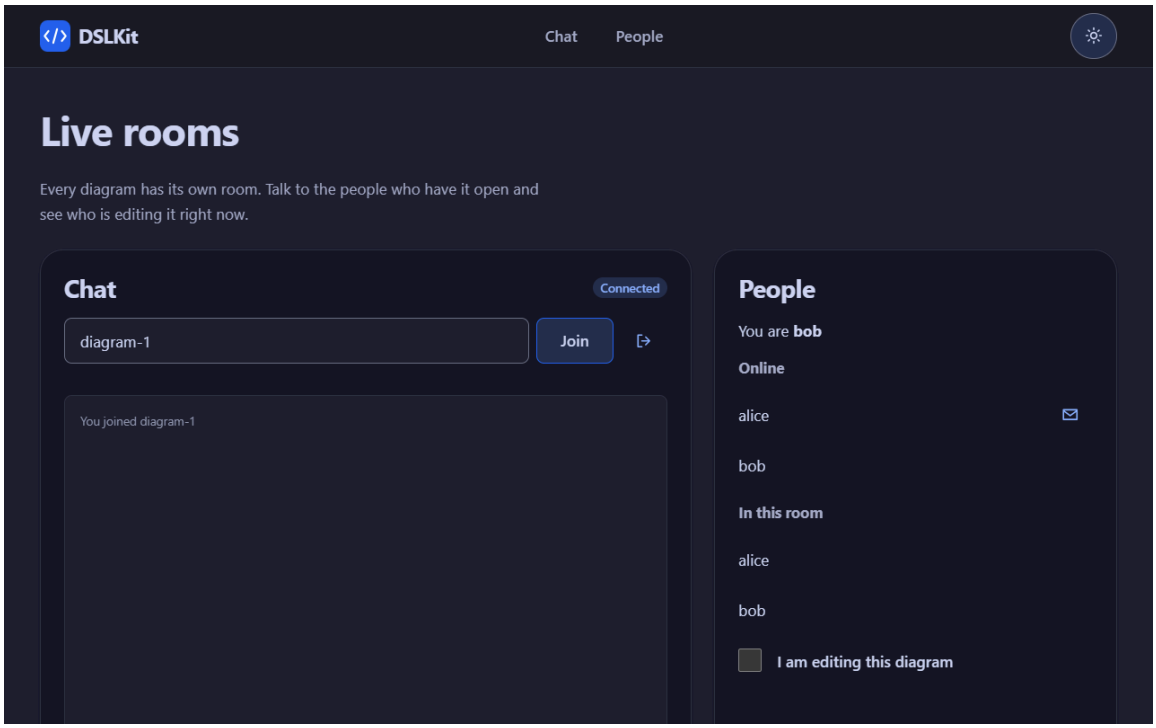


Рисунок 3 – Окно Боба после входа в комнату диаграммы

На рисунке 4 показано: окно Алисы: сообщение Боба пришло в комнату.

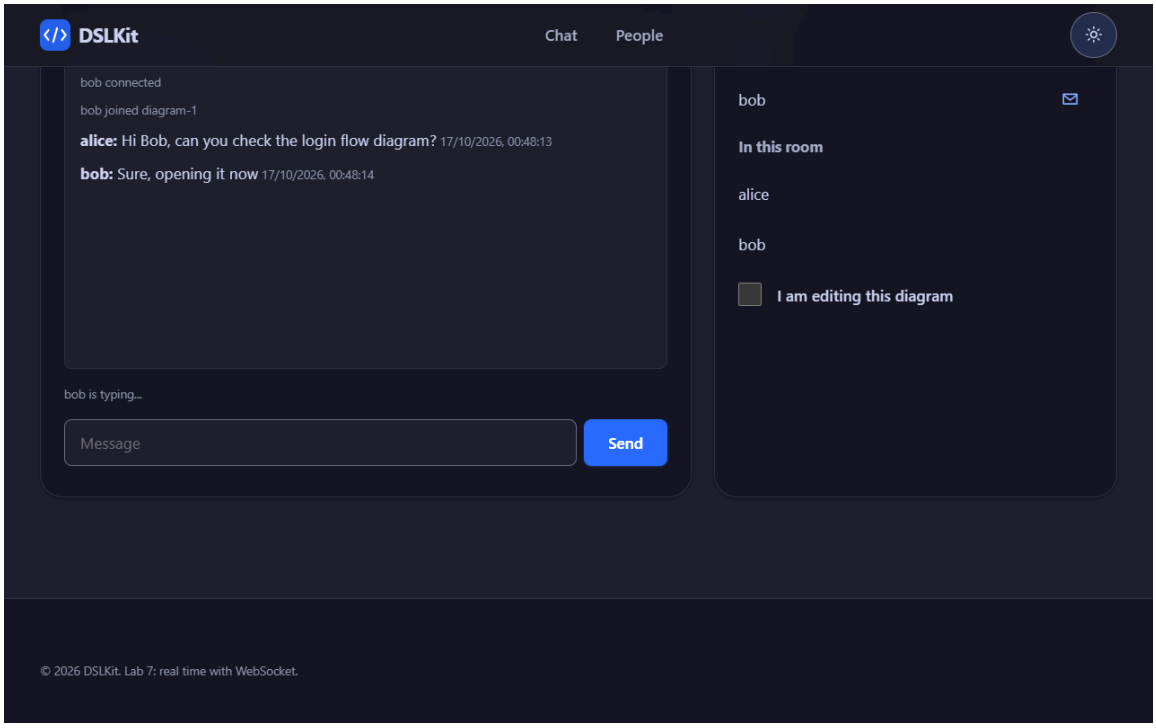


Рисунок 4 – Окно Алисы: сообщение Боба пришло в комнату

На рисунке 5 показано: окно Боба: свои и чужие сообщения в комнате.

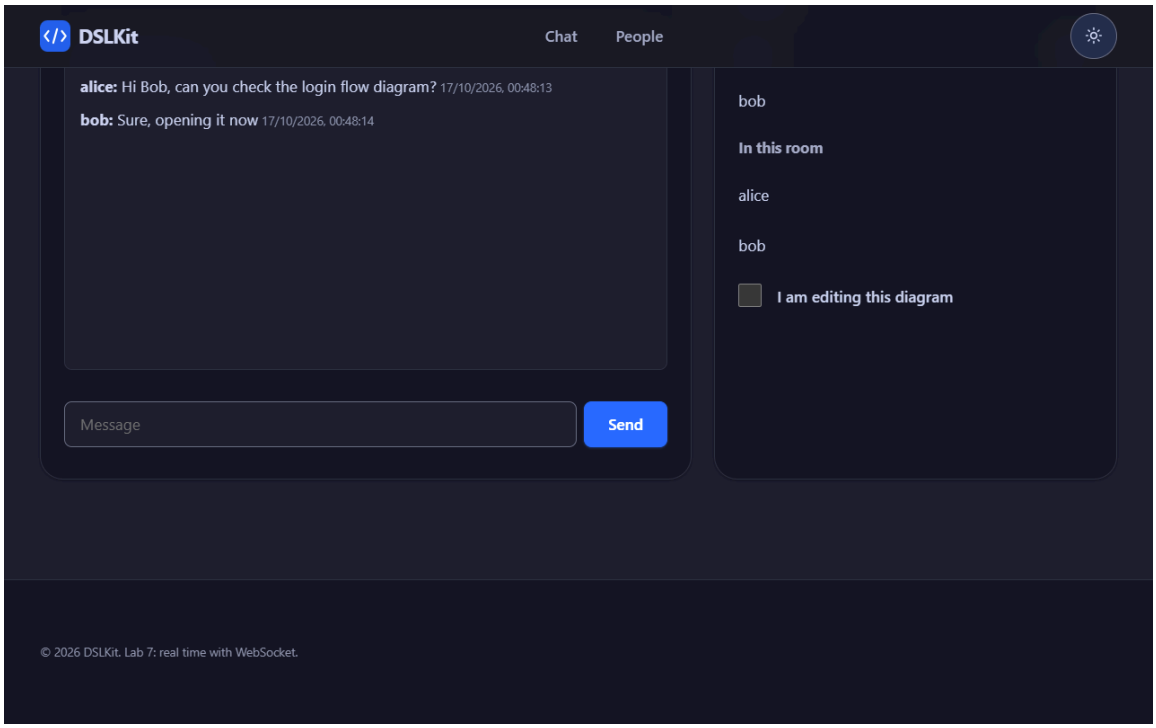


Рисунок 5 – Окно Боба: свои и чужие сообщения в комнате

На рисунке 6 показано: окно Кэрол в другой комнате: сообщения первой комнаты не приходят.

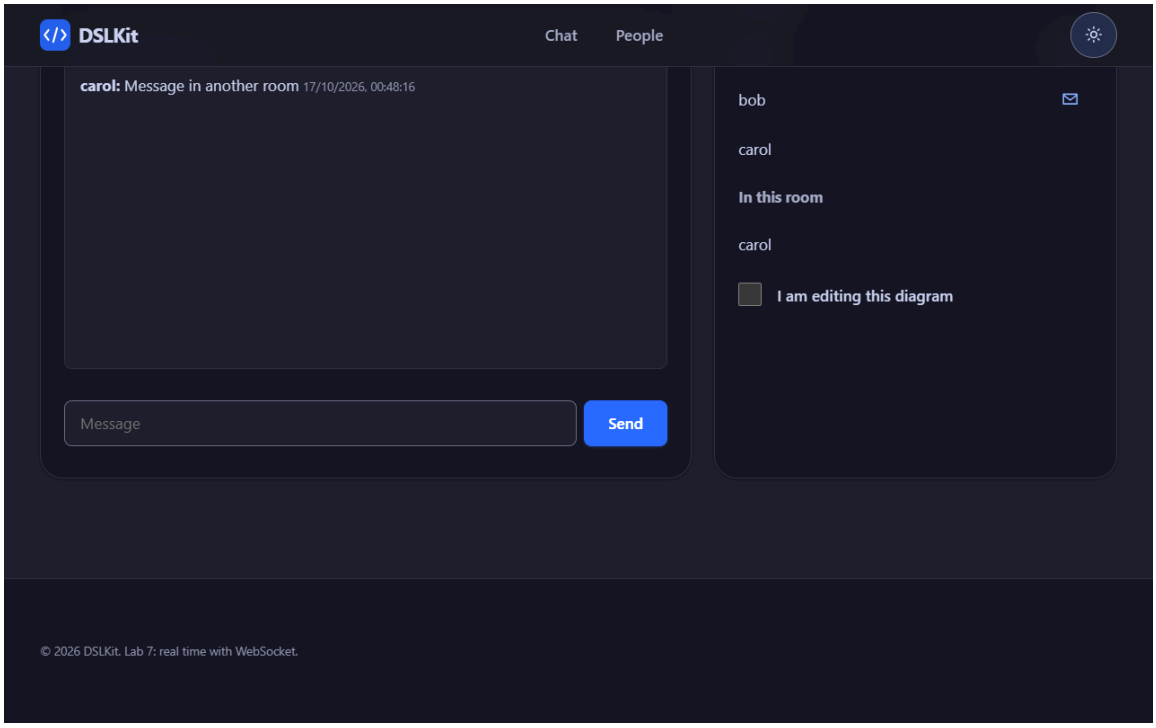


Рисунок 6 – Окно Кэрол в другой комнате: сообщения первой комнаты не приходят

На рисунке 7 показано: окно Алисы: сообщения другой комнаты не попали в её ленту.

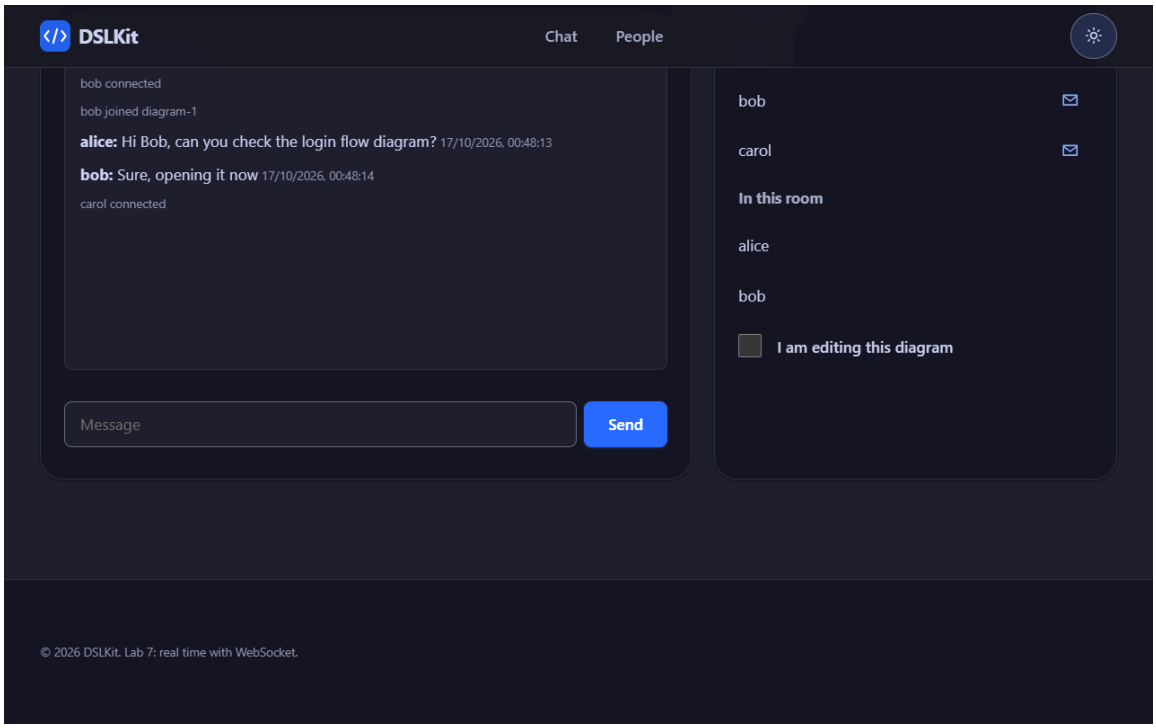


Рисунок 7 – Окно Алисы: сообщения другой комнаты не попали в её ленту

На рисунке 8 показано: окно Боба: список показывает, кто редактирует диаграмму.

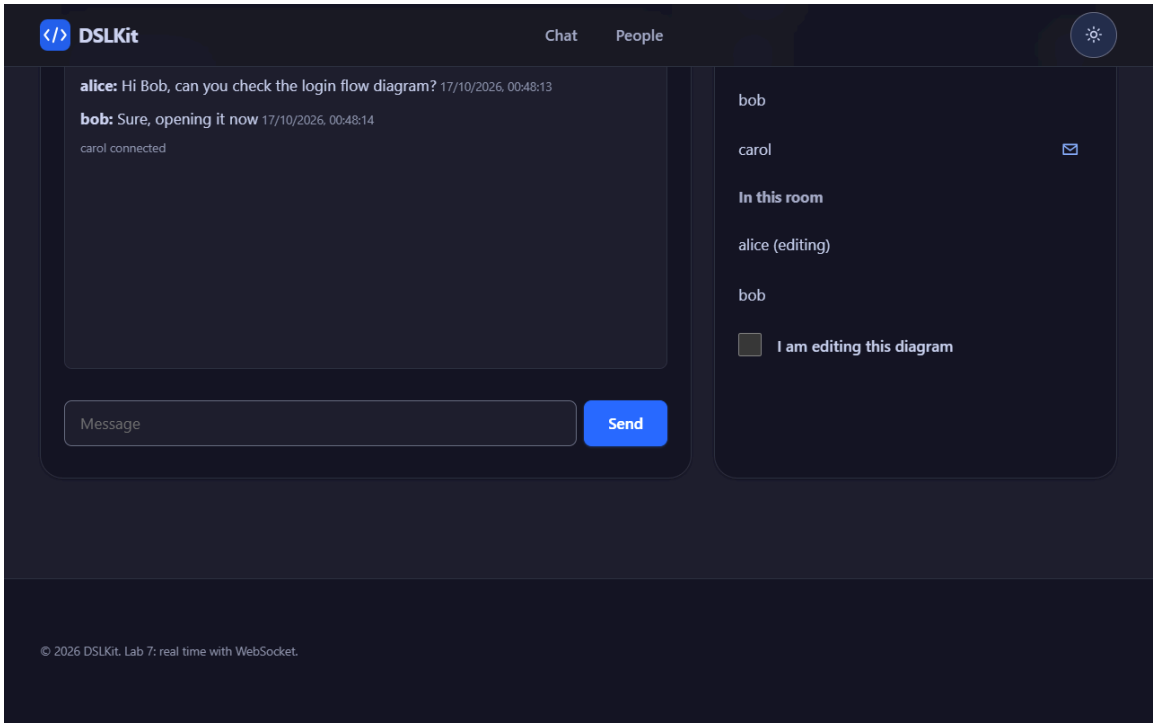


Рисунок 8 – Окно Боба: список показывает, кто редактирует диаграмму

На рисунке 9 показано: окно Боба: ошибка проверки при пустом сообщении.

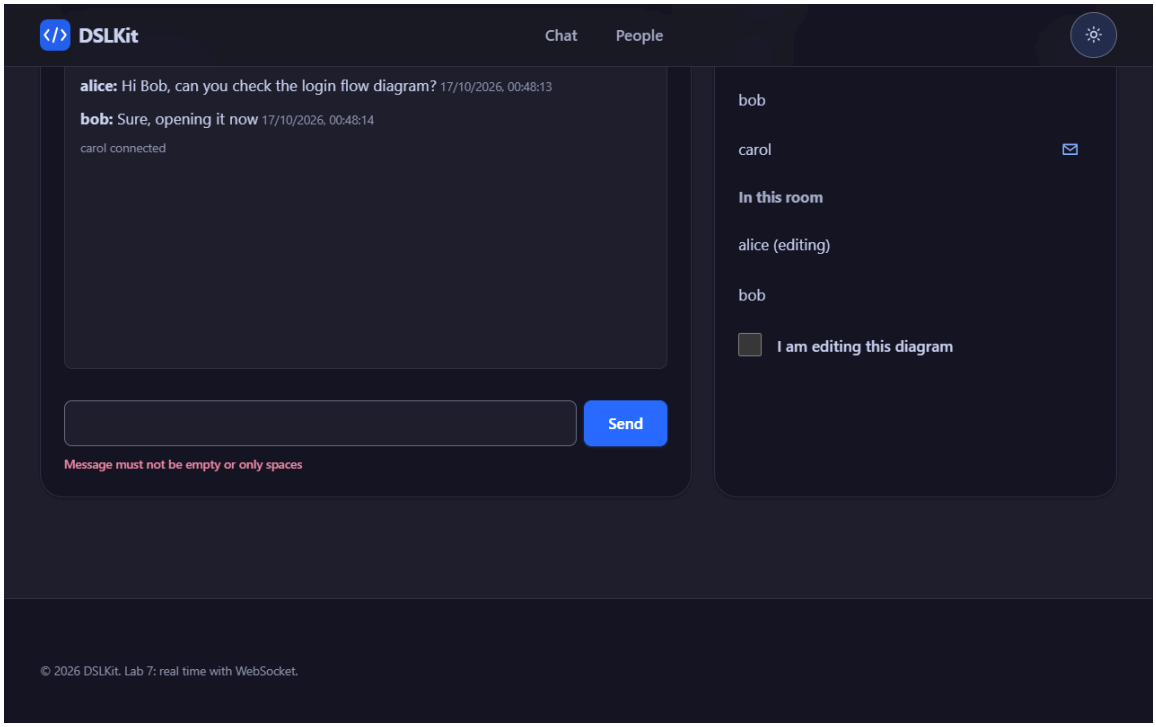


Рисунок 9 – Окно Боба: ошибка проверки при пустом сообщении

На рисунке 10 показано: окно Алисы: уведомление о выходе Боба из комнаты.

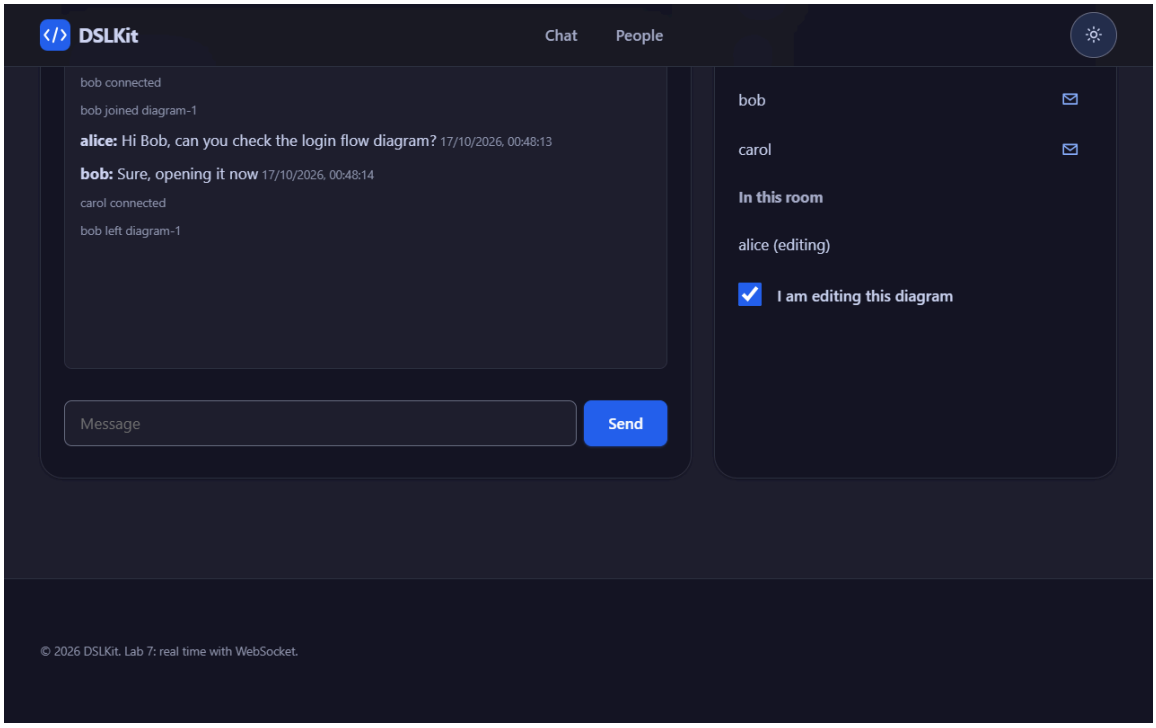


Рисунок 10 – Окно Алисы: уведомление о выходе Боба из комнаты

На рисунке 11 показано: окно Алисы: уведомление об отключении Боба.

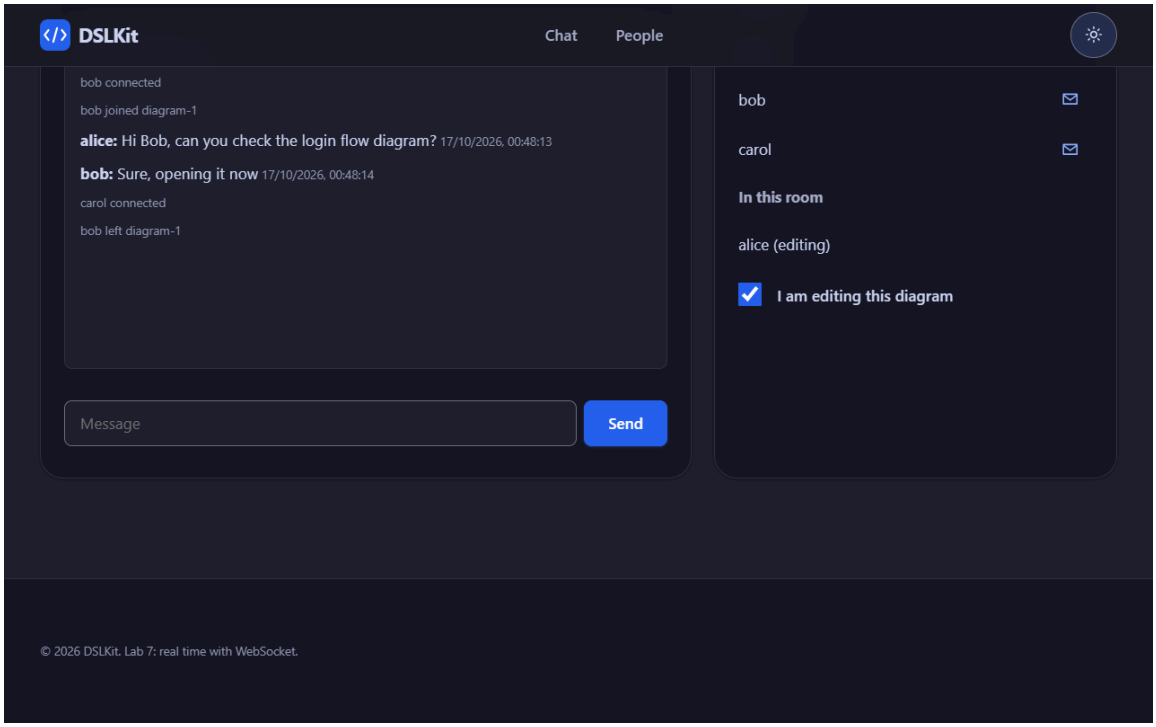


Рисунок 11 – Окно Алисы: уведомление об отключении Боба

3.6 Проверка вводимых значений и ограничения

Все ограничения вынесены в отдельный файл `limits.json`: длина имени пользователя (24), название комнаты (40), длина сообщения (500), размер истории (50), частота сообщений (5 за 2 секунды) и размер пакета. Сервер читает файл при запуске и отдаёт его клиенту по адресу `/limits.json`, поэтому клиент и сервер используют одни и те же числа.

Проверяются три случая. Пустое сообщение отклоняется. Сообщение из одних пробелов после удаления крайних пробелов становится пустым и отклоняется так же. Сообщение длиннее предела отклоняется, а ровно предельной длины принимается. Клиент проверяет пробельный текст до отправки и ограничивает поле атрибутом `maxLength`, сервер повторяет проверку, потому что клиенту доверять нельзя. Ошибка возвращается в acknowledgement и показывается под формой.

Те же правила защищают сервер от перегрузки. Частота всех событий ограничена на каждом сокете (40 за 2 секунды), пользователь может войти не более чем в 5 комнат, в комнате не больше 50 человек, а на сервере не больше 200 подключений. Число комнат с историей тоже ограничено, чтобы выдуманные клиентом названия не занимали память бесконечно. Из текста сообщения удаляются управляющие символы и символы смены направления письма, а имя пользователя считается занятым независимо от регистра букв. Пакет больше 10 000 байт закрывает только соединение отправителя.

Подключение проверяется по заголовку `Origin`. Страница другого сайта не может открыть сокет от имени посетителя, потому что сервер принимает только собственный адрес и адреса из переменной `ALLOWED_ORIGINS`. Значение `*` для CORS не используется. Ответы сервера содержат заголовки `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options` и `Referrer-Policy`, а несуществующий адрес и сбой сервера открывают оформленные страницы 404 и 500 без текста ошибки.

Каждое сообщение получает метку времени `at` в момент получения сервером. Клиент показывает дату и время с секундами рядом с автором, поэтому два сообщения не выглядят одинаково, а новый участник видит время и в сообщениях из истории.

3.7 Ответы на контрольные вопросы

9. Как отправить приватное сообщение конкретному пользователю по его socket.id?

У каждого соединения есть уникальный `socket.id`. Сервер находит сокет в `io.sockets.sockets.get(id)` и вызывает у него `emit`, тогда текст получает только адресат. Короткая запись `io.to(id).emit(...)` работает потому, что каждый сокет автоматически входит в комнату со своим id. В работе событие `private:message` принимает `to` и текст, а отправителю возвращает acknowledgement с копией сообщения. Если сокета с таким id нет, отправитель получает ошибку «User not found».

10. Как в Socket.IO реализовать подтверждение получения сообщения (acknowledgement)?
    Последним аргументом `emit` передаётся функция: `socket.emit('chat:message', data, (reply) => {...})`. На другой стороне обработчик события принимает эту функцию третьим параметром и вызывает её с ответом. Так клиент узнаёт, что сервер принял сообщение или отклонил его, и получает данные, например историю при входе в комнату. В работе все события с ответом возвращают объект `{ ok, error }`.
11. Как масштабировать Socket.IO-приложение на несколько серверов (например, с использованием Redis)? (теоретически)
    Каждый процесс знает только о своих соединениях, поэтому рассылка в комнату не дойдёт до клиентов на другом сервере. Адаптер Redis (`@socket.io/redis-adapter`) пересылает события между процессами через Redis Pub/Sub, и `io.to(room).emit` доходит до всех. Нужен и «липкий» балансировщик, который закрепляет клиента за сервером на время handshake при long polling. Данные вроде истории и списка людей придётся перенести из памяти процесса в общее хранилище.

Листинг server.js приведён в приложении А. Листинг app.js приведён в приложении Б. Листинг socket.js приведён в приложении В. Листинг validate.js приведён в приложении Г. Листинг history.js приведён в приложении Д. Листинг rate-limit.js приведён в приложении Е. Листинг client.js приведён в приложении Ж. Листинг index.html приведён в приложении И.

ЗАКЛЮЧЕНИЕ

Для DSLKit реализован сервер на Express и Socket.IO с клиентом на HTML и JavaScript. Пользователи обмениваются сообщениями в реальном времени, получают уведомления о подключении, отключении, входе в комнату и выходе, а для каждой диаграммы видят список людей и тех, кто её редактирует. Дополнительно сделаны история с хранилищем в MongoDB или в памяти, вход по подписанному токену, личные сообщения, индикатор набора текста, подтверждения и ограничение частоты. 102 автоматических тестов подтверждает поведение каждого события. Ограничения вынесены в `limits.json`, у каждого сообщения есть метка времени. Запуск с настоящей MongoDB и скриншоты двух окон в работу не включены.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг server.js

```
const { createServer } = require('./src/app');

try {
  process.loadEnvFile();
} catch {
  // .env is optional, defaults apply
}

const port = Number(process.env.PORT) || 3000;
// With MONGODB_URI the chat history is kept in MongoDB and surv
ives a restart; without it, in memory.
const start = async () => {
  let history;
  let client;
  if (process.env.MONGODB_URI) {
    const { MongoClient } = require('mongodb');
    const { createMongoHistory } = require('./src/history');
    client = await MongoClient.connect(process.env.MONGODB_URI,
{ serverSelectionTimeoutMS: 5000 });
    history = createMongoHistory(client.db().collection('message
s'));
  }
  if (!process.env.TOKEN_SECRET) console.warn('TOKEN_SECRET is n
ot set, the local development secret is used');
  const { httpServer, io } = createServer({ history });
  httpServer.listen(port, () => console.log(`Server running on h
ttp://localhost:${port}`));
  return { io, client };
};

// Ctrl+C and service stops close the sockets first, so clients
see a clean disconnect and reconnect later.
start()
  .then(({ io, client }) => {
    const stop = () => io.close(() => (client ? client.close() :
Promise.resolve()).finally(() => process.exit(0)));
    process.on('SIGINT', stop);
    process.on('SIGTERM', stop);
  })
  .catch((error) => {
    console.error('Cannot start the server:', error.message);
    process.exit(1);
  });
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг app.js

```
const http = require('node:http');
const path = require('node:path');
const express = require('express');
const { Server } = require('socket.io');
const { attachSocket } = require('./socket');
const { limits, cleanName } = require('./validate');
const { parseAllowed, isOriginAllowed } = require('./origin');
const { issueToken } = require('./token');

const publicDir = path.join(__dirname, '..', 'public');

// Headers for every HTTP answer. The script and style policy fo
rbids inline code, so an
// injected tag cannot run even if some text ever reached the pa
ge unescaped.
const securityHeaders = (req, res, next) => {
  res.set({
    'Content-Security-Policy':
      "default-src 'self'; script-src 'self'; style-src 'self';
img-src 'self' data:; connect-src 'self' ws: wss:; object-src 'n
one'; base-uri 'none'; frame-ancestors 'none'; form-action 'sel
f'",
    'X-Content-Type-Options': 'nosniff',
    'X-Frame-Options': 'DENY',
    'Referrer-Policy': 'no-referrer',
  });
  next();
};

// Builds the HTTP server with the static client and Socket.IO o
n top of it.
// The server is returned unstarted so tests can listen on a fre
e port.
// `allowedOrigins` lists extra sites (besides the server's own
address) that may open a socket.
// The token secret comes from TOKEN_SECRET; the default below i
s for local runs and tests only.
// `requireToken` (REQUIRE_TOKEN=1) refuses sockets that come wi
thout a login token.
const DEFAULT_SECRET = 'local-development-secret-do-not-use-in-p
roduction';

const createServer = ({
  allowedOrigins = parseAllowed(process.env.ALLOWED_ORIGINS),
  tokenSecret = process.env.TOKEN_SECRET || DEFAULT_SECRET,
  requireToken = process.env.REQUIRE_TOKEN === '1',
```

```
  setup,
  ...options
} = {}) => {
  // The built-in secret is public, so a production server must
get its own.
  if (process.env.NODE_ENV === 'production' && tokenSecret === D
EFAULT_SECRET) throw new Error('TOKEN_SECRET is required when NO
DE_ENV is production');
  // A short secret can be guessed offline from one token, so pr
oduction needs at least 32 characters (the same rule as the JWT
secret of lab 3).
  if (process.env.NODE_ENV === 'production' && String(tokenSecre
t).length < 32) throw new Error('TOKEN_SECRET must be at least 3
2 characters when NODE_ENV is production');
  const app = express();
  app.disable('x-powered-by');
  app.use(securityHeaders);
  app.use(express.static(publicDir));
  app.get('/limits.json', (req, res) => res.json(limits));
  app.get('/health', (req, res) => res.json({ status: 'ok' }));
  // Login: a valid name gets a signed token that the socket han
dshake accepts instead of a plain name.
  app.post('/login', express.json({ limit: '1kb' }), (req, res)
=> {
    const username = cleanName(req.body?.username);
    if (!username) return res.status(400).json({ error: `A usern
ame of 1-${limits.nameMaxLength} letters, digits, dots, dashes o
r underscores is required` });
    res.json({ username, token: issueToken(username, tokenSecre
t) });
  });
  // Known paths with another method answer 405 and say which me
thods they accept.
  app.all('/login', (req, res) => res.set('Allow', 'POST').statu
s(405).type('text').send('Method not allowed'));
  app.all('/health', (req, res) => res.set('Allow', 'GET').statu
s(405).type('text').send('Method not allowed'));
  setup?.(app);

  // 404 and 500 pages use the same styles as the rest of the si
te; the error text never reaches the browser.
  app.use((req, res) => res.status(404).sendFile('404.html', { r
oot: publicDir }));
  // eslint-disable-next-line no-unused-vars
  app.use((err, req, res, next) => {
    const status = err.status >= 400 && err.status < 500 ? err.s
tatus : 500;
    if (status === 500) return res.status(500).sendFile('500.htm
l', { root: publicDir });
    res.status(status).type('text').send('Bad request');
  });
```

```
  const httpServer = http.createServer(app);
  const io = new Server(httpServer, {
    // Without extra origins no CORS headers are sent at all: on
ly the page served by this server can talk to it.
    cors: allowedOrigins.length ? { origin: allowedOrigins } : f
alse,
    allowRequest: (req, callback) => callback(null, isOriginAllo
wed(req.headers.origin, req.headers.host, allowedOrigins)),
    maxHttpBufferSize: limits.maxPacketBytes,
  });
  attachSocket(io, { ...options, tokenSecret, requireToken });
  return { app, httpServer, io };
};

module.exports = { createServer };
```

ПРИЛОЖЕНИЕ В

(обязательное)

Листинг socket.js

```
const { cleanName, cleanRoom, cleanText, limits } = require('./v
alidate');
const { createHistory } = require('./history');
const { createLimiter } = require('./rate-limit');
const { verifyToken } = require('./token');
const { emitLight } = require('./slow-clients');

const fail = (ack, error) => {
  if (typeof ack === 'function') ack({ ok: false, error });
};
const succeed = (ack, data = {}) => {
  if (typeof ack === 'function') ack({ ok: true, ...data });
};

// The state shared by every connection. `rooms` maps a diagram
room to the users inside it,
// so the "who is editing" list needs no extra queries.
const createHub = (io, opts) => {
  // `history` can be injected (for example the MongoDB one); th
e default keeps messages in memory.
  const history = opts.history ?? createHistory(opts.historyLimi
t, opts.historyMaxRooms);
  const rooms = new Map();
  const online = new Map(); // socket id -> { id, username }
  const perAddress = new Map(); // client address -> number of o
pen sockets
  let messageSeq = 0;
  const usersOf = (room) => [...(rooms.get(room)?.values() ??
[])];
  const onlineList = () => [...online.values()];
  return {
    io,
    opts,
    history,
    rooms,
    online,
    usersOf,
    onlineList,
    publishPresence: (room) => io.to(room).emit('presence:updat
e', { room, users: usersOf(room) }),
    makeMessage: (user, text, extra) => ({ id: ++messageSeq, fro
m: user.username, fromId: user.id, text, at: Date.now(), ...extr
a }),
    perAddress,
    isTaken: (username) => onlineList().some((u) => u.username.t
oLowerCase() === username.toLowerCase()),
```

```
  };
};

// Handshake: the name comes from a signed login token (`auth.to
ken`, issued by POST /login) or, when
// tokens are not required, from `auth.username`. Every later ev
ent knows who sent it.
const createHandshake = (hub) => (socket, next) => {
  const { opts } = hub;
  const { token, username: plain } = socket.handshake.auth ??
{};
  let given = plain;
  if (token !== undefined) {
    given = verifyToken(token, opts.tokenSecret);
    if (!given) return next(new Error('Invalid or expired token,
log in again'));
  } else if (opts.requireToken) {
    return next(new Error('A login token is required'));
  }
  const username = cleanName(given);
  if (!username) return next(new Error(`A username of 1-${limit
s.nameMaxLength} letters, digits, dots, dashes or underscores is
required`));
  if (hub.isTaken(username)) return next(new Error('This usernam
e is already taken'));
  if (hub.online.size >= opts.maxUsers) return next(new Error('T
he server is full, try again later'));
  // One client cannot take all the places of the server.
  if ((hub.perAddress.get(socket.handshake.address) ?? 0) >= opt
s.maxPerAddress) return next(new Error('Too many connections fro
m your address'));
  socket.data.username = username;
  next();
};

// Room membership and presence of one connected socket. `conn`
holds what belongs to this socket only.
const createRoomActions = (hub, conn) => {
  const { io, opts, history, rooms } = hub;
  const { socket, user, joined } = conn;

  const leave = (room) => {
    joined.delete(room);
    socket.leave(room);
    rooms.get(room)?.delete(socket.id);
    if (rooms.get(room)?.size === 0) rooms.delete(room);
    io.to(room).emit('room:left', { room, username: user.usernam
e, at: Date.now() });
    io.to(room).emit('typing', { room, username: user.username,
typing: false });
    hub.publishPresence(room);
  };
```

```
  const join = async (payload, ack) => {
    const room = cleanRoom(payload?.room);
    if (!room) return fail(ack, 'Invalid room name');
    if (joined.has(room)) return fail(ack, 'Already in this roo
m');
    if (joined.size >= opts.maxRoomsPerUser) return fail(ack, `Y
ou can be in at most ${opts.maxRoomsPerUser} rooms`);
    if ((rooms.get(room)?.size ?? 0) >= opts.maxRoomMembers) ret
urn fail(ack, 'This room is full');
    let past;
    try {
      past = await history.list(room);
    } catch (error) {
      console.error('history unavailable:', error.message);
      return fail(ack, 'History is unavailable, try again late
r');
    }
    if (joined.has(room) || !socket.connected) return fail(ack,
'Already in this room');
    joined.add(room);
    socket.join(room);
    if (!rooms.has(room)) rooms.set(room, new Map());
    rooms.get(room).set(socket.id, user);
    socket.to(room).emit('room:joined', { room, username: user.u
sername, at: Date.now() });
    hub.publishPresence(room);
    succeed(ack, { room, history: past, users: hub.usersOf(room)
});
  };

  const leaveRoom = (payload, ack) => {
    const room = cleanRoom(payload?.room);
    if (!room || !joined.has(room)) return fail(ack, 'Not in thi
s room');
    leave(room);
    succeed(ack, { room });
  };

  // "Who is editing": a user marks the diagram of a room as bei
ng edited right now.
  const editing = (payload, ack) => {
    const room = cleanRoom(payload?.room);
    if (!room || !joined.has(room)) return fail(ack, 'Join the r
oom first');
    user.editing = Boolean(payload.editing);
    hub.publishPresence(room);
    succeed(ack, { editing: user.editing });
  };

  const typing = (payload) => {
    const room = cleanRoom(payload?.room);
```

```
    if (room && joined.has(room)) emitLight(io, room, socket.id,
'typing', { room, username: user.username, typing: Boolean(paylo
ad.typing) }, opts.slowQueueMax);
  };

  return { leave, join, leaveRoom, editing, typing };
};

// Chat messages: to a room and private.
const createMessageActions = (hub, conn) => {
  const { io, history } = hub;
  const { socket, user, joined, allowed } = conn;

  const toRoom = async (payload, ack) => {
    const room = cleanRoom(payload?.room);
    const text = cleanText(payload?.text);
    if (!room || !joined.has(room)) return fail(ack, 'Join the r
oom first');
    if (!text) return fail(ack, `Message must be 1-${limits.text
MaxLength} characters`);
    if (!allowed()) return fail(ack, 'Too many messages, slow do
wn');
    const message = hub.makeMessage(user, text, { room });
    try {
      await history.add(room, message);
    } catch (error) {
      console.error('message was not saved:', error.message);
      return fail(ack, 'The message could not be saved, try agai
n later');
    }
    io.to(room).emit('chat:message', message);
    succeed(ack, { id: message.id });
  };

  // Private message by socket id, delivered only to the target
and echoed to the sender.
  // The sender is always taken from the connection, never from
the payload.
  const toUser = (payload, ack) => {
    const text = cleanText(payload?.text);
    const target = typeof payload?.to === 'string' ? io.sockets.
sockets.get(payload.to) : undefined;
    if (!target) return fail(ack, 'User not found');
    if (target.id === socket.id) return fail(ack, 'You cannot se
nd a private message to yourself');
    if (!text) return fail(ack, `Message must be 1-${limits.text
MaxLength} characters`);
    if (!allowed()) return fail(ack, 'Too many messages, slow do
wn');
    const message = hub.makeMessage(user, text, { to: target.id,
toName: target.data.username });
    target.emit('private:message', message);
```

```
    succeed(ack, { message });
  };

  return { toRoom, toUser };
};

const handleConnection = (hub, socket) => {
  const { opts } = hub;
  const user = { id: socket.id, username: socket.data.username,
editing: false };
  const conn = { socket, user, joined: new Set(), allowed: creat
eLimiter(opts.rateMax, opts.rateWindowMs) };
  const eventAllowed = createLimiter(opts.eventRateMax, opts.eve
ntRateWindowMs);
  hub.online.set(socket.id, { id: user.id, username: user.userna
me });
  const address = socket.handshake.address;
  hub.perAddress.set(address, (hub.perAddress.get(address) ?? 0)
+ 1);

  // Flood guard for every event (join, typing, editing...); cha
t messages have their own, stricter limit.
  socket.use((packet, next) => {
    if (eventAllowed()) return next();
    fail(packet.at(-1), 'Too many requests, slow down');
  });

  // Handlers of one socket run one after another, so a leave se
nt right after a join waits for the history.
  let tail = Promise.resolve();
  const on = (event, handler) =>
    socket.on(event, (...args) => {
      tail = tail.then(() => handler(...args)).catch((error) =>
console.error(event, error.message));
    });

  socket.emit('welcome', { id: user.id, username: user.username,
online: hub.onlineList() });
  socket.broadcast.emit('user:online', { id: user.id, username:
user.username, at: Date.now() });

  const roomActions = createRoomActions(hub, conn);
  const messageActions = createMessageActions(hub, conn);
  on('room:join', roomActions.join);
  on('room:leave', roomActions.leaveRoom);
  on('chat:message', messageActions.toRoom);
  on('private:message', messageActions.toUser);
  on('typing', roomActions.typing);
  on('editing', roomActions.editing);

  // Cleanup: leave every room, forget the user and tell the oth
ers. Nothing is kept per socket afterwards.
```

```
  socket.on('disconnect', () => {
    [...conn.joined].forEach(roomActions.leave);
    hub.online.delete(socket.id);
    const left = (hub.perAddress.get(address) ?? 1) - 1;
    if (left > 0) hub.perAddress.set(address, left);
    else hub.perAddress.delete(address);
    socket.broadcast.emit('user:offline', { id: user.id, usernam
e: user.username, at: Date.now() });
  });
};

// Wires all Socket.IO events.
const attachSocket = (io, overrides = {}) => {
  const hub = createHub(io, { ...limits, ...overrides });
  io.use(createHandshake(hub));
  io.on('connection', (socket) => handleConnection(hub, socke
t));
};

module.exports = { attachSocket };
```

ПРИЛОЖЕНИЕ Г

(обязательное)

Листинг validate.js

```
const limits = require('../limits.json');

const NAME_RE = new RegExp(String.raw`^[\p{L}\p{N}_.-]{1,${limit
s.nameMaxLength}}$`, 'u');
const ROOM_RE = new RegExp(`^[a-z0-9][a-z0-9_-]{0,${limits.roomM
axLength - 1}}$`, 'i');
// C0/C1 control characters except new line and tab, plus the bi
directional overrides that can flip the text around a name.
const HIDDEN_RE = /[\u0000-\u0008\u000b-\u001f\u007f-\u009f\u202
a-\u202e\u2066-\u2069]/g;

const cleanName = (value) => (typeof value === 'string' && NAME_
RE.test(value.trim()) ? value.trim() : null);

// Room names are case-insensitive: "Diagram-1" and "diagram-1"
are the same room.
const cleanRoom = (value) => (typeof value === 'string' && ROOM_
RE.test(value) ? value.toLowerCase() : null);

// A whitespace-only text trims to an empty string and is reject
ed like an empty one.
const cleanText = (value) => {
  if (typeof value !== 'string' || value.length > limits.textMax
Length * 2) return null;
  const text = value.replace(HIDDEN_RE, '').trim();
  return text.length > 0 && text.length <= limits.textMaxLength
? text : null;
};

module.exports = { cleanName, cleanRoom, cleanText, limits };
```

ПРИЛОЖЕНИЕ Д
(обязательное)
Листинг history.js

```
// Message history, one capped list per room. The socket handler
s only use `add` and `list` and
// wait for both with `await`, so any store with these two metho
ds can be plugged in: the in-memory
// one below, or the MongoDB one. The number of rooms is capped
in memory, otherwise a client
// inventing room names could grow the memory without limit.
const createHistory = (limit = 50, maxRooms = 200) => {
  const rooms = new Map();
  return {
    add(room, message) {
      const list = rooms.get(room) ?? [];
      list.push(message);
      if (list.length > limit) list.shift();
      rooms.delete(room); // re-insert so the most recently used
room is the last one
      rooms.set(room, list);
      if (rooms.size > maxRooms) rooms.delete(rooms.keys().next
().value);
    },
    list(room) {
      return [...(rooms.get(room) ?? [])];
    },
  };
};

// History in a MongoDB collection (the object returned by `db.c
ollection('messages')` of the
// official driver). Only insertOne and find(...).sort(...).limi
t(...).toArray() are used, so tests
// can pass a small fake collection and need no database.
const createMongoHistory = (collection, limit = 50) => ({
  async add(room, message) {
    await collection.insertOne({ ...message, room });
  },
  async list(room) {
    const newest = await collection.find({ room }).sort({ _id: -
1 }).limit(limit).toArray();
    // Newest first in the query, oldest first for the client; t
he database id stays inside.
    return newest.reverse().map(({ _id, ...message }) => messag
e);
  },
});

module.exports = { createHistory, createMongoHistory };
```

ПРИЛОЖЕНИЕ Е
(обязательное)
Листинг rate-limit.js

```
// Sliding window: at most `max` events per `windowMs` for one s
ocket.
const createLimiter = (max, windowMs, now = Date.now) => {
  let stamps = [];
  return () => {
    const t = now();
    stamps = stamps.filter((s) => t - s < windowMs);
    if (stamps.length >= max) return false;
    stamps.push(t);
    return true;
  };
};

module.exports = { createLimiter };
```

ПРИЛОЖЕНИЕ Ж

(обязательное)

Листинг client.js

```
/* global io */
const $ = (id) => document.getElementById(id);

let socket = null;
let me = null;
let currentRoom = null;
let privateTo = null; // { id, name } while a private message is
being written
let online = []; // [{ id, username }]
let members = []; // [{ id, username, editing }]
let limits = { nameMaxLength: 24, roomMaxLength: 40, textMaxLeng
th: 500 };
let lastTypingSent = 0;
let typingStopTimer;
const typingUsers = new Map(); // username -> timer that hides t
he "is typing" note

// Every piece of text from the network goes through textConten
t, never parsed as markup,
// so a name or a message can not inject HTML. A test scans this
file for the HTML-parsing APIs.
const el = (tag, className, text) => {
  const node = document.createElement(tag);
  if (className) node.className = className;
  if (text !== undefined) node.textContent = text;
  return node;
};

// Full date and time, so two messages never look the same.
const time = (at) => new Date(at).toLocaleString();

const nameRe = () => new RegExp(String.raw`^[\p{L}\p{N}_.-]
{1,${limits.nameMaxLength}}$`, 'u');
const roomRe = () => new RegExp(`^[a-z0-9][a-z0-9_-]{0,${limits.
roomMaxLength - 1}}$`, 'i');

const setText = (id, message = '') => ($(id).textContent = messa
ge);
const check = (id) => (reply) => setText(id, reply?.ok ? '' : (r
eply?.error ?? 'No answer from the server'));

const addLine = ({ className = '', author, at, text, room }) =>
{
  const li = el('li', `message ${className}`.trim());
  if (room) li.dataset.room = room;
  if (author) li.append(el('strong', '', `${author}: `));
```

```
  li.append(el('span', '', text));
  if (at) {
    const stamp = el('time', '', ` ${time(at)}`);
    stamp.dateTime = new Date(at).toISOString();
    li.append(stamp);
  }
  const log = $('log');
  const stick = log.scrollTop + log.clientHeight >= log.scrollHe
ight - 8;
  log.append(li);
  while (log.children.length > 300) log.firstChild.remove(); //
the page keeps a bounded number of lines
  if (stick) log.scrollTop = log.scrollHeight;
};
const system = (text) => addLine({ className: 'message--system',
text });
const addChat = (m) => addLine({ author: m.from, at: m.at, text:
m.text, room: m.room });

const renderPeople = (list, users, withActions) => {
  list.replaceChildren();
  if (!users.length) list.append(el('li', 'people__item people__
empty', 'Nobody'));
  users.forEach((u) => {
    const li = el('li', 'people__item');
    li.append(el('span', '', u.editing ? `${u.username} (editin
g)` : u.username));
    if (withActions && u.id !== me?.id) {
      const button = el('button', 'button button--quiet button--
icon live__icon');
      button.type = 'button';
      button.dataset.tip = 'Private message';
      button.setAttribute('aria-label', `Write a private message
to ${u.username}`);
      button.append(envelope());
      button.addEventListener('click', () => setTarget({ id: u.i
d, name: u.username }));
      li.append(button);
    }
    list.append(li);
  });
};

// A small envelope drawn with DOM calls, so no markup string is
parsed.
const envelope = () => {
  const ns = 'http://www.w3.org/2000/svg';
  const svg = document.createElementNS(ns, 'svg');
  svg.setAttribute('class', 'icon');
  svg.setAttribute('viewBox', '0 0 20 20');
  svg.setAttribute('fill', 'none');
  svg.setAttribute('stroke', 'currentColor');
```

```
  svg.setAttribute('stroke-width', '1.5');
  svg.setAttribute('aria-hidden', 'true');
  svg.setAttribute('focusable', 'false');
  const path = document.createElementNS(ns, 'path');
  path.setAttribute('d', 'M3 5h14v10H3zM3 6l7 5 7-5');
  svg.append(path);
  return svg;
};

const renderOnline = () => renderPeople($('online'), online, tru
e);
const renderMembers = () => renderPeople($('members'), members,
false);

const setTarget = (target) => {
  privateTo = target;
  $('target').hidden = !target;
  $('target-name').textContent = target ? target.name : '';
  $('text').focus();
};

const setConnection = (text) => ($('conn').textContent = text);

const showTyping = () => {
  const names = [...typingUsers.keys()];
  $('typing').textContent = names.length ? `${names.join(', ')}
${names.length > 1 ? 'are' : 'is'} typing...` : '';
};

const joinRoom = (room, { rejoin = false } = {}) =>
  socket.emit('room:join', { room }, (reply) => {
    check('room-error')(reply);
    if (!reply?.ok) {
      if (rejoin) currentRoom = null;
      return;
    }
    currentRoom = reply.room;
    $('room').value = reply.room;
    members = reply.users;
    renderMembers();
    // Re-render the room history instead of appending it, so a
rejoin after a lost connection never shows a message twice.
    $('log')
      .querySelectorAll('[data-room]')
      .forEach((node) => node.remove());
    reply.history.forEach(addChat);
    system(rejoin ? `Back in ${reply.room}` : `You joined ${repl
y.room}`);
  });

const bind = (s) => {
  s.on('welcome', ({ id, username, online: list }) => {
```

```
    me = { id, username };
    online = list;
    $('me').textContent = username;
    $('login').hidden = true;
    $('workspace').hidden = false;
    renderOnline();
    setConnection('Connected');
  });
  s.on('user:online', (u) => {
    online = [...online.filter((x) => x.id !== u.id), { id: u.i
d, username: u.username }];
    renderOnline();
    system(`${u.username} connected`);
  });
  s.on('user:offline', (u) => {
    online = online.filter((x) => x.id !== u.id);
    if (privateTo?.id === u.id) setTarget(null);
    renderOnline();
    system(`${u.username} disconnected`);
  });
  s.on('room:joined', ({ username, room }) => system(`${usernam
e} joined ${room}`));
  s.on('room:left', ({ username, room }) => system(`${username}
left ${room}`));
  s.on('presence:update', ({ room, users }) => {
    if (room !== currentRoom) return;
    members = users;
    renderMembers();
  });
  s.on('chat:message', (m) => m.room === currentRoom && addChat
(m));
  s.on('private:message', (m) => addLine({ className: 'message--
private', author: `${m.from} (private)`, at: m.at, text: m.text
}));
  s.on('typing', ({ room, username, typing }) => {
    if (room !== currentRoom) return;
    clearTimeout(typingUsers.get(username));
    typingUsers.delete(username);
    // If the "stopped" event is lost, the note disappears by it
self after three seconds.
    if (typing)
      typingUsers.set(
        username,
        setTimeout(() => (typingUsers.delete(username), showTypi
ng()), 3000),
      );
    showTyping();
  });
  s.on('disconnect', (reason) => {
    setConnection('Reconnecting...');
    setText('status', reason === 'io server disconnect' ? 'The s
erver closed the connection.' : 'Connection lost, trying to reco
```

```
nnect...');
    typingUsers.forEach(clearTimeout);
    typingUsers.clear();
    showTyping();
  });
  // Fires after the first connection and after every automatic
reconnection: the room is joined again.
  s.on('connect', () => {
    setText('status');
    if (currentRoom) joinRoom(currentRoom, { rejoin: true });
  });
  s.on('connect_error', (error) => {
    if (!me) {
      setText('login-error', error.message);
      s.close();
      return;
    }
    // After a restart the old connection may still be registere
d for a moment: try again instead of giving up.
    setText('status', error.message);
    if (!s.active) setTimeout(() => s.connect(), 2000);
  });
};

$('login').addEventListener('submit', async (event) => {
  event.preventDefault();
  const username = $('username').value.trim();
  if (!username) return setText('login-error', 'Enter a name, sp
aces alone are not a name');
  if (!nameRe().test(username)) return setText('login-error', `U
se 1-${limits.nameMaxLength} letters, digits, dots, dashes or un
derscores`);
  setText('login-error');
  socket?.close(); // a second submit never leaves a second conn
ection behind
  try {
    // The server signs the name into a token; the socket then p
roves who it is with the token.
    const reply = await fetch('/login', { method: 'POST', header
s: { 'Content-Type': 'application/json' }, body: JSON.stringify
({ username }) });
    const data = await reply.json();
    if (!reply.ok) return setText('login-error', data.error ??
'Login failed');
    socket = io({ auth: { token: data.token } });
    bind(socket);
  } catch {
    setText('login-error', 'The server is not reachable');
  }
});

$('room-form').addEventListener('submit', (event) => {
```

```
  event.preventDefault();
  const room = $('room').value.trim();
  if (!roomRe().test(room)) return setText('room-error', `Room n
ame: 1-${limits.roomMaxLength} letters, digits, dashes or unders
cores`);
  setText('room-error');
  joinRoom(room.toLowerCase());
});

$('leave').addEventListener('click', () => {
  socket.emit('room:leave', { room: currentRoom }, (reply) => {
    check('room-error')(reply);
    if (!reply?.ok) return;
    system(`You left ${currentRoom}`);
    currentRoom = null;
    members = [];
    renderMembers();
    $('editing').checked = false;
    typingUsers.clear();
    showTyping();
  });
});

$('editing').addEventListener('change', (event) => socket.emit
('editing', { room: currentRoom, editing: event.target.checked
}, check('status')));
$('target-clear').addEventListener('click', () => setTarget(nul
l));

$('text').addEventListener('input', () => {
  if (!currentRoom) return;
  const now = Date.now();
  if (now - lastTypingSent > 1500) {
    lastTypingSent = now;
    socket.emit('typing', { room: currentRoom, typing: true });
  }
  clearTimeout(typingStopTimer);
  typingStopTimer = setTimeout(() => {
    lastTypingSent = 0;
    socket.emit('typing', { room: currentRoom, typing: false });
  }, 1000);
});

$('chat-form').addEventListener('submit', (event) => {
  event.preventDefault();
  const text = $('text').value;
  if (!text.trim()) return setText('status', 'Message must not b
e empty or only spaces');
  if (text.trim().length > limits.textMaxLength) return setText
('status', `Message must be at most ${limits.textMaxLength} char
acters`);
  if (!privateTo && !currentRoom) return setText('status', 'Join
```

```
a room first');
  const done = (reply) => {
    check('status')(reply);
    if (!reply?.ok) return;
    $('text').value = '';
    if (reply.message)
      addLine({ className: 'message--private', author: `You to
${reply.message.toName} (private)`, at: reply.message.at, text:
reply.message.text });
  };
  if (privateTo) socket.emit('private:message', { to: privateTo.
id, text }, done);
  else socket.emit('chat:message', { room: currentRoom, text },
done);
});

// The same limits as on the server: limits.json is served at /l
imits.json.
fetch('/limits.json')
  .then((r) => r.json())
  .then((loaded) => {
    limits = { ...limits, ...loaded };
    $('text').maxLength = limits.textMaxLength;
    $('username').maxLength = limits.nameMaxLength;
    $('room').maxLength = limits.roomMaxLength;
  })
  .catch(() => {}); // the defaults above stay in place when the
file can not be loaded
```

ПРИЛОЖЕНИЕ И
(обязательное)
Листинг index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-s
cale=1" />
    <meta name="description" content="DSLKit live rooms: chat, w
ho is online and who is editing a diagram." />
    <meta name="theme-color" content="#2563eb" />
    <title>Live rooms — DSLKit</title>
    <link rel="icon" href="favicon.svg" type="image/svg+xml" />
    <link rel="stylesheet" href="dslkit.css" />
    <link rel="stylesheet" href="style.css" />
    <script src="theme.js"></script>
  </head>
  <body class="page">
    <a class="skip-link" href="#main">Skip to the main content</
a>
    <header class="header">
      <div class="header__inner">
        <a class="header__logo" href="/">
          <img class="header__logo-image" src="logo.svg" alt=""
width="32" height="32" />
          <span class="header__logo-text">DSLKit</span>
        </a>
        <input class="burger__input" type="checkbox" id="burger-
toggle" />
        <label class="burger__button" for="burger-toggle">
          <span class="burger__label">Menu</span>
          <span class="burger__icon" aria-hidden="true"></span>
        </label>
        <nav class="nav" aria-label="Main navigation">
          <ul class="nav__list">
            <li class="nav__item"><a class="nav__link" href="#ch
at-title">Chat</a></li>
            <li class="nav__item"><a class="nav__link" href="#pe
ople-title">People</a></li>
          </ul>
        </nav>
        <div class="header__tools">
          <input class="theme__input" type="checkbox" id="theme-
toggle" />
          <label class="theme__button" for="theme-toggle" title
="Dark theme"
            ><svg class="theme__icon theme__icon--moon" viewBox
="0 0 24 24" width="20" height="20" aria-hidden="true" focusable
```

```
="false">
              <path fill="none" stroke="currentColor" stroke-wid
th="2" stroke-linecap="round" stroke-linejoin="round" d="M21 12.
8A9 9 0 1 1 11.2 3a7 7 0 0 0 9.8 9.8Z" /></svg
            ><svg class="theme__icon theme__icon--sun" viewBox
="0 0 24 24" width="20" height="20" aria-hidden="true" focusable
="false">
              <circle cx="12" cy="12" r="4" fill="none" stroke
="currentColor" stroke-width="2" />
              <path
                fill="none"
                stroke="currentColor"
                stroke-width="2"
                stroke-linecap="round"
                d="M12 2v2M12 20v2M4.9 4.9l1.4 1.4M17.7 17.7l1.4
1.4M2 12h2M20 12h2M4.9 19.1l1.4-1.4M17.7 6.3l1.4-1.4"
              /></svg
            ><span class="theme__label">Dark theme</span></label
          >
        </div>
      </div>
    </header>

    <main class="main" id="main">
      <h1 class="live__title">Live rooms</h1>
      <p class="live__lead">Every diagram has its own room. Talk
to the people who have it open and see who is editing it right n
ow.</p>

      <form id="login" class="form" novalidate>
        <div class="form__field">
          <label class="form__label" for="username">Username</la
bel>
          <input class="form__input" id="username" name="usernam
e" maxlength="24" autocomplete="off" aria-describedby="username-
hint login-error" required />
          <p class="form__hint" id="username-hint">Up to 24 lett
ers, digits, dots, dashes or underscores.</p>
          <p class="form__error" id="login-error" role="alert">
</p>
        </div>
        <div class="actions">
          <button class="button button--primary" type="submit">C
onnect</button>
        </div>
      </form>

      <div id="workspace" class="live" hidden>
        <section class="live__people form" aria-labelledby="peop
le-title">
          <h2 class="live__heading" id="people-title">People</h2
>
```

```
          <p class="live__me">You are <strong id="me"></strong>
</p>
          <h3 class="live__subheading">Online</h3>
          <ul id="online" class="people"></ul>
          <h3 class="live__subheading">In this room</h3>
          <ul id="members" class="people"></ul>
          <div class="form__field form__field--checkbox">
            <input class="form__checkbox" type="checkbox" id="ed
iting" />
            <label class="form__label" for="editing">I am editin
g this diagram</label>
          </div>
        </section>

        <section class="live__chat form" aria-labelledby="chat-t
itle">
          <div class="live__bar">
            <h2 class="live__heading" id="chat-title">Chat</h2>
            <span class="badge" id="conn" role="status">Connecte
d</span>
          </div>
          <form id="room-form" class="live__row" novalidate>
            <label class="visually-hidden" for="room">Diagram ro
om</label>
            <input class="form__input" id="room" value="diagram-
1" maxlength="40" autocomplete="off" />
            <button class="button" type="submit">Join</button>
            <button class="button button--quiet button--icon liv
e__icon" type="button" id="leave" data-tip="Leave the room" aria
-label="Leave the room">
              <svg class="icon" aria-hidden="true" focusable="fa
lse" viewBox="0 0 20 20" fill="none" stroke="currentColor" strok
e-width="1.5">
                <path d="M8 4H4v12h4M12 6l4 4-4 4M16 10H8" />
              </svg>
            </button>
          </form>
          <p class="form__error" id="room-error" role="alert"></
p>

          <ul id="log" class="log" aria-live="polite" aria-label
="Messages"></ul>
          <p class="form__hint" id="typing" aria-live="polite">
</p>

          <form id="chat-form" class="live__send" novalidate>
            <p class="live__target" id="target" hidden>
              <span>Private to <strong id="target-name"></strong
></span>
              <button class="button button--quiet button--icon l
ive__icon" type="button" id="target-clear" data-tip="Write to th
e room" aria-label="Write to the room instead">
```

```
                <svg class="icon" aria-hidden="true" focusable
="false" viewBox="0 0 20 20" fill="none" stroke="currentColor" s
troke-width="1.5">
                  <path d="M5 5l10 10M15 5L5 15" />
                </svg>
              </button>
            </p>
            <div class="live__row">
              <label class="visually-hidden" for="text">Message
</label>
              <input class="form__input" id="text" maxlength="50
0" autocomplete="off" placeholder="Message" />
              <button class="button button--primary" type="submi
t">Send</button>
            </div>
            <p class="form__error" id="status" role="alert"></p>
          </form>
        </section>
      </div>
    </main>

    <footer class="footer">
      <div class="footer__inner">
        <p class="footer__copy"><small>© 2026 DSLKit. Lab 7: rea
l time with WebSocket.</small></p>
      </div>
    </footer>
    <script src="/socket.io/socket.io.js"></script>
    <script src="client.js"></script>
  </body>
</html>
```